

COMP1549: Advanced Programming

Design by Contract

Dr Taimoor Khan

WWW

- [Personal](#)
- Operational Lead - NCSC accredited [ACE-CSR](#)
- Deputy Director - NCSC accredited [ACE-CSE](#)
- [RITICS Fellow](#) – Imperial College London

19th February 2026 - Week 6

Design by Contract

Problem: write a program to add two numbers.

```
public int add(int x, int y){ return x+y; }
```

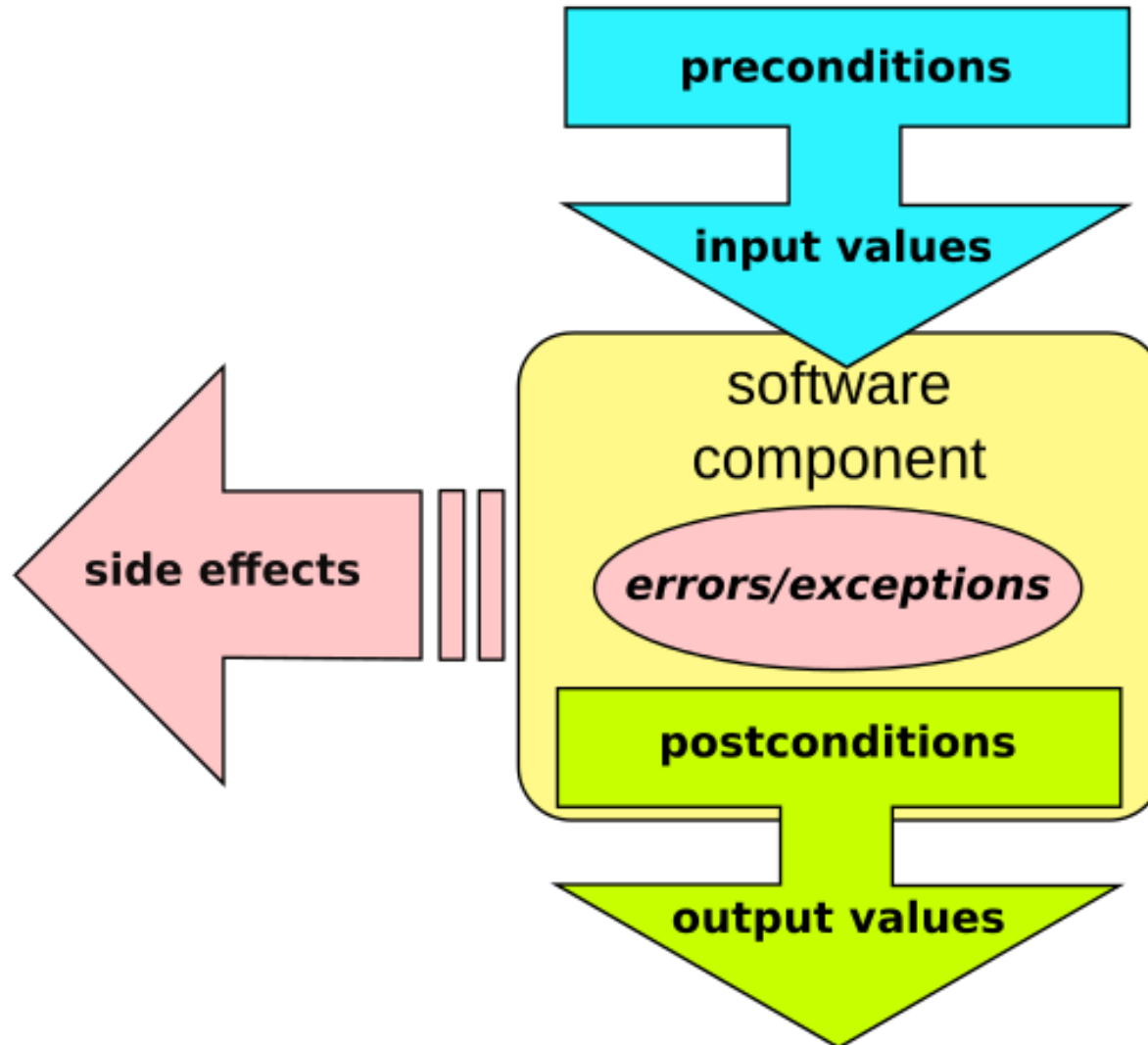
- Does the above program solve the problem?
- Do we understand the complete program?
- Is this program correct?

Design by Contract (DBC)

- A way of proving:
 - Details of program responsibilities/requirements, e.g.,
 - ❖ Does the program terminate?
 - ❖ Under what conditions does the program work?
 - ❖ What result does the program return?
 - ❖ How the program computes the result?

DBC helps develop program that are correct by construction

Design by contract



DBC annotations

- **Preconditions** of methods

- A Boolean expression which is assumed true when the method gets called

- **Postconditions** of methods

- A Boolean expression which the caller can assume to be true when the method returns

- **Loop invariant**

- consistency conditions for loops
- must hold for all iterations

- **Loop variant**

- termination condition of loops
- must decrease after each iteration
- must be ≥ 0

Software contract

```
/*@ requires x >= 0.0;  
  @ ensures JMLDouble.approximatelyEqualTo(x,  
    @      \result * \result, eps);  
  @*/  
public static double sqrt(double x) { ... }
```

	Obligations	Rights
Client	Passes non-negative number	Gets square root approximation
Implementor	Computes and returns square root	Assumes argument is non-negative

Pre and Postconditions

● Definition

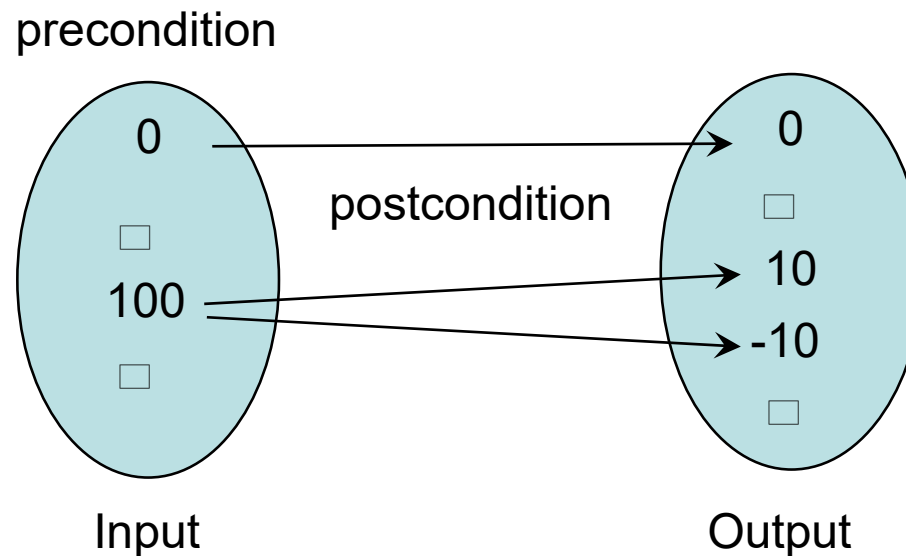
- A method's *precondition* says what must be true to call it.
- A method's *normal postcondition* says what is true when it returns normally (i.e., without throwing an exception).
- A method's *exceptional postcondition* says what is true when a method throws an exception.

```
/*@ signals (IllegalArgumentException e) x < 0;  
  @*/
```


Relational model of methods

- Can think of a method as a relation:

Inputs $\leftrightarrow$ Outputs



Contracts as documentation

- ◉ For each method say:
 - What it requires (if constraints on inputs), and
 - What it ensures (as a result).
- ◉ Contracts are:
 - More abstract than code,
 - Not necessarily constructive,
 - Often machine checkable, so can help with debugging, and
 - Machine checkable contracts can always be up-to-date and certified.

More advantages of contracts

- Blame assignment
 - Who is to blame if:
 - ❖ Precondition doesn't hold?
 - ❖ Postcondition doesn't hold?
- Avoids inefficient defensive checks

//@ requires a != null && (* a is sorted *);

```
public static int binarySearch(Thing[] a, Thing x) { ... }
```

JML

- What is it?
 - Stands for “Java Modeling Language”
 - ❖ A formal behavioral interface specification language for Java
 - Design by contract for Java
 - Uses Java 1.4 or later
 - Originally available from www.jmlspecs.org
 - Updated OpenJML available from <https://www.openjml.org>

- JML specifications are contained in annotations, which are comments like:

`//@ ...`

or

`/*@ ...`

`@ ...`

`@*/`

“@” on the beginning of lines are ignored within annotations.

- Question
 - What's the advantage of using annotations?

Informal description

- An informal description looks like:

(* some text describing a property *)

- It is treated as a Boolean value by JML, and
- Allows
 - ❖ Escape from formality, and
 - ❖ Organize English as contracts.

```
public class IMath {  
    /*@ requires (* x is positive *);  
       @ ensures \result >= 0 &&  
       @   (* \result is an int approximation to square root of x *)  
       @*/  
    public static int isqrt(int x) { ... }  
}
```

Exercise

- Write informal contract (i.e., pre and postconditions) for methods of the following class.

<pre>public class Person { private String name; private int weight; public String toString() { return "Person(\" + name + "\", \" + weight + \")"; } public int getWeight() { return weight; } }</pre>	<pre> public void addKgs(int kgs) { weight += kgs; } public Person(String n) { name = n; weight = 0; } }</pre>
---	--

Formal specifications

- Formal assertions are written as Java expressions, but:
 - Cannot have side effects
 - No use of `=`, `++`, `--`, etc., and
 - Can only call *pure* methods.
 - Can use some extensions to Java:

Syntax	Meaning
<code>\result</code>	result of method call
<code>a ==> b</code>	a implies b
<code>a <== b</code>	b implies a
<code>a <==> b</code>	a iff b
<code>a <=!> b</code>	$\neg(a \text{ <==> } b)$
<code>\old(E)</code>	value of E in pre-state

Example

```
// File: Person.refines-java

//@ refine "Person.java"

public class Person {
    private /*@ spec_public non_null @*/ String name;
    private /*@ spec_public @*/ int weight;

    //@ public invariant !name.equals("") && weight >= 0;

    /*@ also
       @ ensures \result != null;
       @*/
    public String toString();

    //@ also ensures \result == weight;
    public int getWeight();

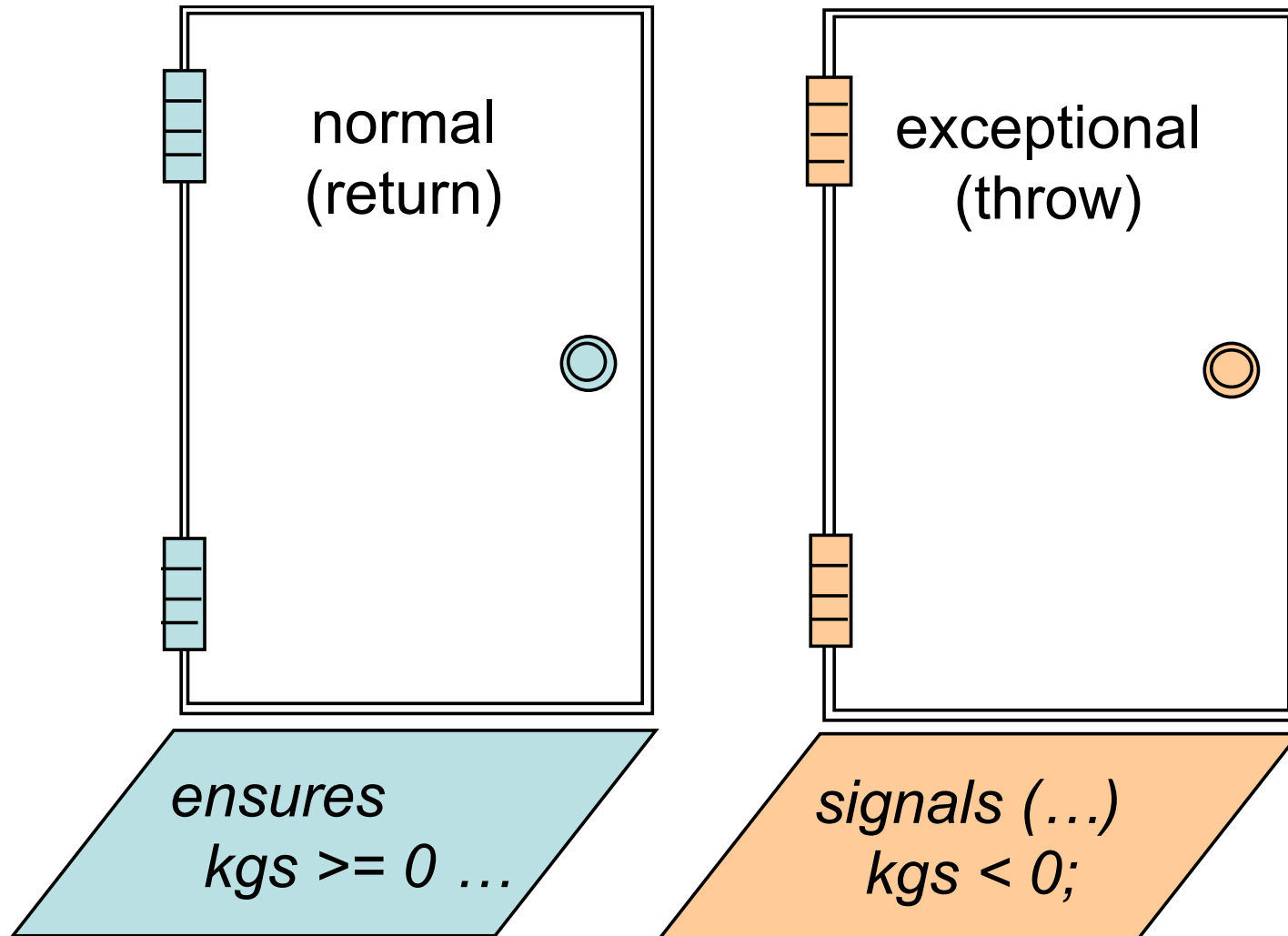
    ...
}
```

Example (Cont.)

```
/*@ also
  @ requires kgs >= 0;
  @ ensures weight == \old(weight) + kgs;
  @ signals (Exception e) kgs < 0 &&
  @           (e instanceof IllegalArgumentException);
  @*/
public void addKgs(int kgs);

/*@ also
  @ requires !n.equals("");
  @ ensures n.equals(name) && weight == 0;
  @*/
public Person(/*@ non_null @*/ String n);
}
```

Meaning of postconditions



Invariant

- Definition

- An *invariant* is a property that is always true of an object's state (when control is not inside the object's methods).

- Invariants allow you to define:

- Acceptable states of an object, and
- Consistency of an object's state.

```
//@ public invariant !name.equals("") && weight >= 0;
```

Exercise

- Formally specify the following method (in Person)

```
public void changeName(String newName) {  
    name = newName;  
}
```

Hint: watch out for the invariant!

Quantifiers

- JML supports several forms of quantifiers

- Universal and existential (`\forall` and `\exists`)
- General quantifiers (`\sum`, `\product`, `\min`, `\max`)
- Numeric quantifier (`\num_of`)

`(\forall Student s; juniors.contains(s); s.getAdvisor() != null)`

`(\forall Student s; juniors.contains(s) ==> s.getAdvisor() != null)`

Example

```
//@ requires a != null;  
//@ ensures \result == -1 => (\forall int i; 0 <= i && i < a.length; a[i]%2 != 0);  
//@ ensures \result != -1 => \result % 2 == 0 &&  
//@ ensures (\exists int j; 0 <= j && j < a.length; a[j] == \result &&  
//@ (\forall int i; j < i && i < a.length; a[i]%2 != 0));
```

```
/*@ pure @*/ int searchEven(int[] a){  
    int num = -1;  
    int index = 0;  
    //@ loop_invariant 0 <= index && index <= a.length;  
    //@ loop_invariant num != -1 ==>  
    //@ (\exists int j; 0 <= j && j < index; a[j] == num && num%2 == 0 &&  
    //@ (\forall int i; 0 <= i && i < index; a[i]%2 == 0 ==> j>=i));  
    //@ decreases a.length - index;  
    while(index < a.length){  
        if(a[index] % 2 == 0){num = a[index]; }  
        index = index+1;  
    }  
    return num;  
}
```


Tools for JML

- ◉ JML compiler (jmlc)
- ◉ JML/Java interpreter (jmlrac)
- ◉ JML/JUnit unit test tool (jmlunit)
- ◉ HTML generator (jmldoc)

- ◉ **Recent initiative OpenJML**
<https://www.openjml.org>

JML Compiler (jmlc)

- Basic usage

```
$ jmlc Person.java
```

produces Person.class

```
$ jmlc -Q *.java
```

produces *.class, quietly

```
$ jmlc -d ../bin Person.java
```

produces ../bin/Person.class

Running code compiled with jmlc

- Must have JML's runtime classes (jmlruntime.jar) in Java's boot class path
- Automatic if you use script jmlrac, e.g.,
\$ jmlrac PersonMain

A Main Program

```
public class PersonMain {  
    public static void main(String[] args) {  
        System.out.println(new Person(null));  
        System.out.println(new Person(""));  
    }  
}
```

Example

```
$ jmlc -Q Person.java
```

```
$ javac PersonMain.java
```

```
$ jmlrac PersonMain
```

```
Exception in thread "main" org.jmlspecs.jmlrac.runtime.JMLEntryPreconditionError
```

```
: by method Person.Person regarding specifications at
```

```
File "Person.refines-java", line 52, character 20 when
```

```
  'n' is null
```

```
at org.jmlspecs.samples.jmltutorial.Person.checkPre$$init$$Person(  
    Person.refines-java:1060)
```

```
at org.jmlspecs.samples.jmltutorial.Person.<init>(Person.refines-java:51)
```

```
at org.jmlspecs.samples.jmltutorial.PersonMain.main(PersonMain.java:27)
```

Summary

- JML is a powerful DBC tool for Java
 - For details, visit <https://www.openjml.org>
- Spec# is DBC tool for C# (Java like language)
 - <https://www.rise4fun.com/SpecSharp>
- Why3 is another more advanced DBC tool
 - <https://www.why3.org/try/>

References

- Meyer, Bertrand: *Design by Contract*, Technical Report TR-EI-12/CO, Interactive Software Engineering Inc., 1986
- Slides are adapted from <http://www.eecs.ucf.edu/~leavens/JML//index.shtml>

End of week 6!